2.3 Adversarial training to smooth predictive distributions

Adversarial examples, proposed by Szegedy et al. [55] and extended by Goodfellow et al. [18], are those which are 'close' to the original training examples (e.g. an image that is visually indistinguishable from the original image to humans), but are misclassified by the NN. Goodfellow et al. [18] proposed the *fast gradient sign method* as a fast solution to generate adversarial examples. Given an input $\mathbf{x}$ with target y, and loss $\ell(\theta, \mathbf{x}, y)$ (e.g. $-\log p_\theta(y|\mathbf{x})$), the fast gradient sign method generates an adversarial example as $\mathbf{x}' = \mathbf{x} + \epsilon \operatorname{sign}\big(\nabla_{\mathbf{x}} \ell(\theta, \mathbf{x}, y)\big)$, where ϵ is a small value such that the max-norm of the perturbation is bounded. Intuitively, the adversarial perturbation creates a new training example by adding a perturbation along a direction which the network is likely to increase the loss. Assuming ϵ is small enough, these adversarial examples can be used to augment the original training set by treating $(\mathbf{x}', y)$ as additional training examples. This procedure, referred to as *adversarial training*,[3] was found to improve the classifier's robustness [18].

Interestingly, adversarial training can be interpreted as a computationally efficient solution to smooth the predictive distributions by increasing the likelihood of the target around an ϵ-neighborhood of the observed training examples. Ideally one would want to smooth the predictive distributions along all 2^D directions in $\{1, -1\}^D$; however this is computationally expensive. A random direction might not necessarily increase the loss; however, adversarial training by definition computes the direction where the loss is high and hence is better than a random direction for smoothing predictive distributions. Miyato et al. [44] proposed a related idea called *virtual adversarial training* (VAT), where they picked $\Delta\mathbf{x} = \arg\max_{\Delta\mathbf{x}} \mathrm{KL}\big(p(y|\mathbf{x})||p(y|\mathbf{x} + \Delta\mathbf{x})\big)$; the advantage of VAT is that it does not require knowledge of the true target y and hence can be applied to semi-supervised learning. Miyato et al. [44] showed that distributional smoothing using VAT is beneficial for efficient semi-supervised learning; in contrast, we investigate the use of adversarial training for predictive uncertainty estimation. Hence, our contributions are complementary; one could use VAT or other forms of adversarial training, cf. [33], for improving predictive uncertainty in the semi-supervised setting as well.

2.4 Ensembles: training and prediction

The most popular ensembles use decision trees as the base learners and a wide variety of method have been explored in the literature on ensembles. Broadly, there are two classes of ensembles: *randomization*-based approaches such as random forests [8], where the ensemble members can be trained in parallel without any interaction, and *boosting*-based approaches where the ensemble members are fit sequentially. We focus only on the randomization based approach as it is better suited for distributed, parallel computation. Breiman [8] showed that the generalization error of random forests can be upper bounded by a function of the strength and correlation between individual trees; hence it is desirable to use a *randomization scheme* that de-correlates the predictions of the individual models as well as ensures that the individual models are strong (e.g. high accuracy). One of the popular strategies is *bagging* (a.k.a. bootstrapping), where ensemble members are trained on different bootstrap samples of the original training set. If the base learner lacks intrinsic randomization (e.g. it can be trained efficiently by solving a convex optimization problem), bagging is a good mechanism for inducing diversity. However, if the underlying base learner has multiple local optima, as is the case typically with NNs, the bootstrap can sometimes hurt performance since a base learner trained on a bootstrap sample sees only 63% unique data points.[4] In the literature on decision tree ensembles, Breiman [8] proposed to use a combination of bagging [7] and random subset selection of features at each node. Geurts et al. [16] later showed that bagging is unnecessary if additional randomness can be injected into the random subset selection procedure. Intuitively, using more data for training the base learners helps reduce their bias and ensembling helps reduce the variance.

We used the entire training dataset to train each network since deep NNs typically perform better with more data, although it is straightforward to use a random subsample if need be. We found that random initialization of the NN parameters, along with random shuffling of the data points, was sufficient to obtain good performance in practice. We observed that bagging deteriorated performance in our experiments. Lee et al. [36] independently observed that training on entire dataset with random initialization was better than bagging for deep ensembles, however their goal was to improve

[3] Not to be confused with Generative Adversarial Networks (GANs).

[4] The bootstrap draws N times uniformly with replacement from a dataset with N items. The probability an item is picked at least once is $1 - (1 - 1/N)^N$, which for large N becomes $1 - e^{-1} \approx 0.632$. Hence, the number of unique data points in a bootstrap sample is $0.632 \times N$ on average.

predictive accuracy and not predictive uncertainty. The overall training procedure is summarized in Algorithm 1.

Algorithm 1 Pseudocode of the training procedure for our method

1: ▷ *Let each neural network parametrize a distribution over the outputs, i.e.* $p_\theta(y|\mathbf{x})$. *Use a proper scoring rule as the training criterion* $\ell(\theta, \mathbf{x}, y)$. *Recommended default values are* $M = 5$ *and* $\boldsymbol{\epsilon} = 1\%$ *of the input range of the corresponding dimension (e.g 2.55 if input range is [0,255]).*
2: Initialize $\theta_1, \theta_2, \ldots, \theta_M$ randomly
3: **for** $m = 1 : M$ **do** ▷ *train networks independently in parallel*
4: Sample data point n_m randomly for each net ▷ *single* n_m *for clarity, minibatch in practice*
5: Generate adversarial example using $\mathbf{x}'_{n_m} = \mathbf{x}_{n_m} + \boldsymbol{\epsilon}\, \mathrm{sign}\big(\nabla_{\mathbf{x}_{n_m}} \ell(\theta_m, \mathbf{x}_{n_m}, y_{n_m})\big)$
6: Minimize $\ell(\theta_m, \mathbf{x}_{n_m}, y_{n_m}) + \ell(\theta_m, \mathbf{x}'_{n_m}, y_{n_m})$ w.r.t. θ_m ▷ *adversarial training (optional)*

We treat the ensemble as a uniformly-weighted mixture model and combine the predictions as $p(y|\mathbf{x}) = M^{-1} \sum_{m=1}^{M} p_{\theta_m}(y|\mathbf{x}, \theta_m)$. For classification, this corresponds to averaging the predicted probabilities. For regression, the prediction is a mixture of Gaussian distributions. For ease of computing quantiles and predictive probabilities, we further approximate the ensemble prediction as a Gaussian whose mean and variance are respectively the mean and variance of the mixture. The mean and variance of a mixture $M^{-1} \sum \mathcal{N}\big(\mu_{\theta_m}(\mathbf{x}), \sigma^2_{\theta_m}(\mathbf{x})\big)$ are given by $\mu_*(\mathbf{x}) = M^{-1} \sum_m \mu_{\theta_m}(\mathbf{x})$ and $\sigma^2_*(\mathbf{x}) = M^{-1} \sum_m \big(\sigma^2_{\theta_m}(\mathbf{x}) + \mu^2_{\theta_m}(\mathbf{x})\big) - \mu^2_*(\mathbf{x})$ respectively.

3 Experimental results

3.1 Evaluation metrics and experimental setup

For both classification and regression, we evaluate the negative log likelihood (NLL) which depends on the predictive uncertainty. NLL is a proper scoring rule and a popular metric for evaluating predictive uncertainty [49]. For classification we additionally measure classification accuracy and the Brier score, defined as $BS = K^{-1} \sum_{k=1}^{K} \big(t^*_k - p(y = k|\mathbf{x}^*)\big)^2$ where $t^*_k = 1$ if $k = y^*$, and 0 otherwise. For regression problems, we additionally measured the root mean squared error (RMSE). Unless otherwise specified, we used batch size of 100 and Adam optimizer with fixed learning rate of 0.1 in our experiments. We use the same technique for generating adversarial training examples for regression problems. Goodfellow et al. [18] used a fixed ϵ for all dimensions; this is unsatisfying if the input dimensions have different ranges. Hence, in all of our experiments, we set ϵ to 0.01 times the range of the training data along that particular dimension. We used the default weight initialization in Torch.

3.2 Regression on toy datasets

First, we qualitatively evaluate the performance of the proposed method on a one-dimensional toy regression dataset. This dataset was used by Hernández-Lobato and Adams [24], and consists of 20 training examples drawn as $y = x^3 + \epsilon$ where $\epsilon \sim \mathcal{N}(0, 3^2)$. We used the same architecture as [24].

A commonly used heuristic in practice is to use an ensemble of NNs (trained to minimize MSE), obtain multiple point predictions and use the empirical variance of the networks' predictions as an approximate measure of uncertainty. We demonstrate that this is inferior to learning the variance by training using NLL.[5] The results are shown in Figure 1.

The results clearly demonstrate that (i) learning variance and training using a scoring rule (NLL) leads to improved predictive uncertainty and (ii) ensemble combination improves performance, especially as we move farther from the observed training data.

3.3 Regression on real world datasets

In our next experiment, we compare our method to state-of-the-art methods for predictive uncertainty estimation using NNs on regression tasks. We use the experimental setup proposed by Hernández-Lobato and Adams [24] for evaluating probabilistic backpropagation (PBP), which was also used

[5] See also Appendix A.2 for calibration results on a real world dataset.

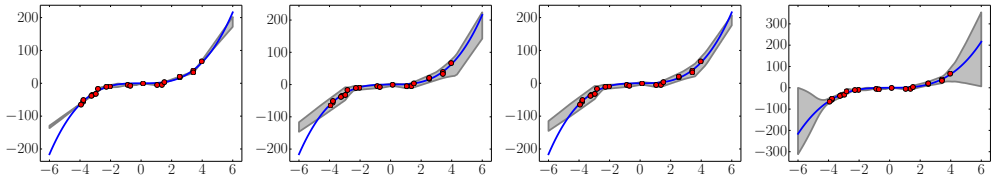

Figure 1: Results on a toy regression task: x-axis denotes x. On the y-axis, the blue line is the *ground truth* curve, the red dots are observed noisy training data points and the gray lines correspond to the predicted mean along with three standard deviations. Left most plot corresponds to empirical variance of 5 networks trained using MSE, second plot shows the effect of training using NLL using a single net, third plot shows the additional effect of adversarial training, and final plot shows the effect of using an ensemble of 5 networks respectively.

by Gal and Ghahramani [15] to evaluate MC-dropout.[6] Each dataset is split into 20 train-test folds, except for the protein dataset which uses 5 folds and the Year Prediction MSD dataset which uses a single train-test split. We use the identical network architecture: 1-hidden layer NN with ReLU nonlinearity [45], containing 50 hidden units for smaller datasets and 100 hidden units for the larger protein and Year Prediction MSD datasets. We trained for 40 epochs; we refer to [24] for further details about the datasets and the experimental protocol. We used 5 networks in our ensemble. Our results are shown in Table 1, along with the PBP and MC-dropout results reported in their respective papers.

Datasets	RMSE			NLL		
	PBP	MC-dropout	Deep Ensembles	PBP	MC-dropout	Deep Ensembles
Boston housing	**3.01 ± 0.18**	**2.97 ± 0.85**	**3.28 ± 1.00**	**2.57 ± 0.09**	**2.46 ± 0.25**	**2.41 ± 0.25**
Concrete	**5.67 ± 0.09**	**5.23 ± 0.53**	**6.03 ± 0.58**	**3.16 ± 0.02**	**3.04 ± 0.09**	**3.06 ± 0.18**
Energy	**1.80 ± 0.05**	**1.66 ± 0.19**	**2.09 ± 0.29**	2.04 ± 0.02	1.99 ± 0.09	**1.38 ± 0.22**
Kin8nm	0.10 ± 0.00	0.10 ± 0.00	**0.09 ± 0.00**	-0.90 ± 0.01	-0.95 ± 0.03	**-1.20 ± 0.02**
Naval propulsion plant	0.01 ± 0.00	0.01 ± 0.00	**0.00 ± 0.00**	-3.73 ± 0.01	-3.80 ± 0.05	**-5.63 ± 0.05**
Power plant	**4.12 ± 0.03**	**4.02 ± 0.18**	**4.11 ± 0.17**	2.84 ± 0.01	**2.80 ± 0.05**	**2.79 ± 0.04**
Protein	4.73 ± 0.01	**4.36 ± 0.04**	4.71 ± 0.06	2.97 ± 0.00	2.89 ± 0.01	**2.83 ± 0.02**
Wine	**0.64 ± 0.01**	**0.62 ± 0.04**	**0.64 ± 0.04**	0.97 ± 0.01	**0.93 ± 0.06**	**0.94 ± 0.12**
Yacht	**1.02 ± 0.05**	**1.11 ± 0.38**	**1.58 ± 0.48**	1.63 ± 0.02	1.55 ± 0.12	**1.18 ± 0.21**
Year Prediction MSD	8.88 ± NA	**8.85 ± NA**	8.89 ± NA	3.60 ± NA	3.59 ± NA	**3.35 ± NA**

Table 1: Results on regression benchmark datasets comparing RMSE and NLL. See Table 2 for results on variants of our method.

We observe that our method outperforms (or is competitive with) existing methods in terms of NLL. On some datasets, we observe that our method is slightly worse in terms of RMSE. We believe that this is due to the fact that our method optimizes for NLL (which captures predictive uncertainty) instead of MSE. Table 2 in Appendix A.1 reports additional results on variants of our method, demonstrating the advantage of using an ensemble as well as learning variance.

3.4 Classification on MNIST, SVHN and ImageNet

Next we evaluate the performance on classification tasks using MNIST and SVHN datasets. Our goal is not to achieve the state-of-the-art performance on these problems, but rather to evaluate the effect of adversarial training as well as the number of networks in the ensemble. To verify if adversarial training helps, we also include a baseline which picks a random signed vector. For MNIST, we used an MLP with 3-hidden layers with 200 hidden units per layer and ReLU non-linearities with batch normalization. For MC-dropout, we added dropout after each non-linearity with 0.1 as the dropout rate.[7] Results are shown in Figure 2(a). We observe that adversarial training and increasing the number of networks in the ensemble significantly improve performance in terms of both classification accuracy as well as NLL and Brier score, illustrating that our method produces well-calibrated uncertainty estimates. Adversarial training leads to better performance than augmenting with random direction. Our method also performs much better than MC-dropout in terms of all the performance measures. Note that augmenting the training dataset with invariances (such as random crop and horizontal flips) is complementary to adversarial training and can potentially improve performance.

[6] We do not compare to VI [19] as PBP and MC-dropout outperform VI on these benchmarks.

[7] We also tried dropout rate of 0.5, but that performed worse.